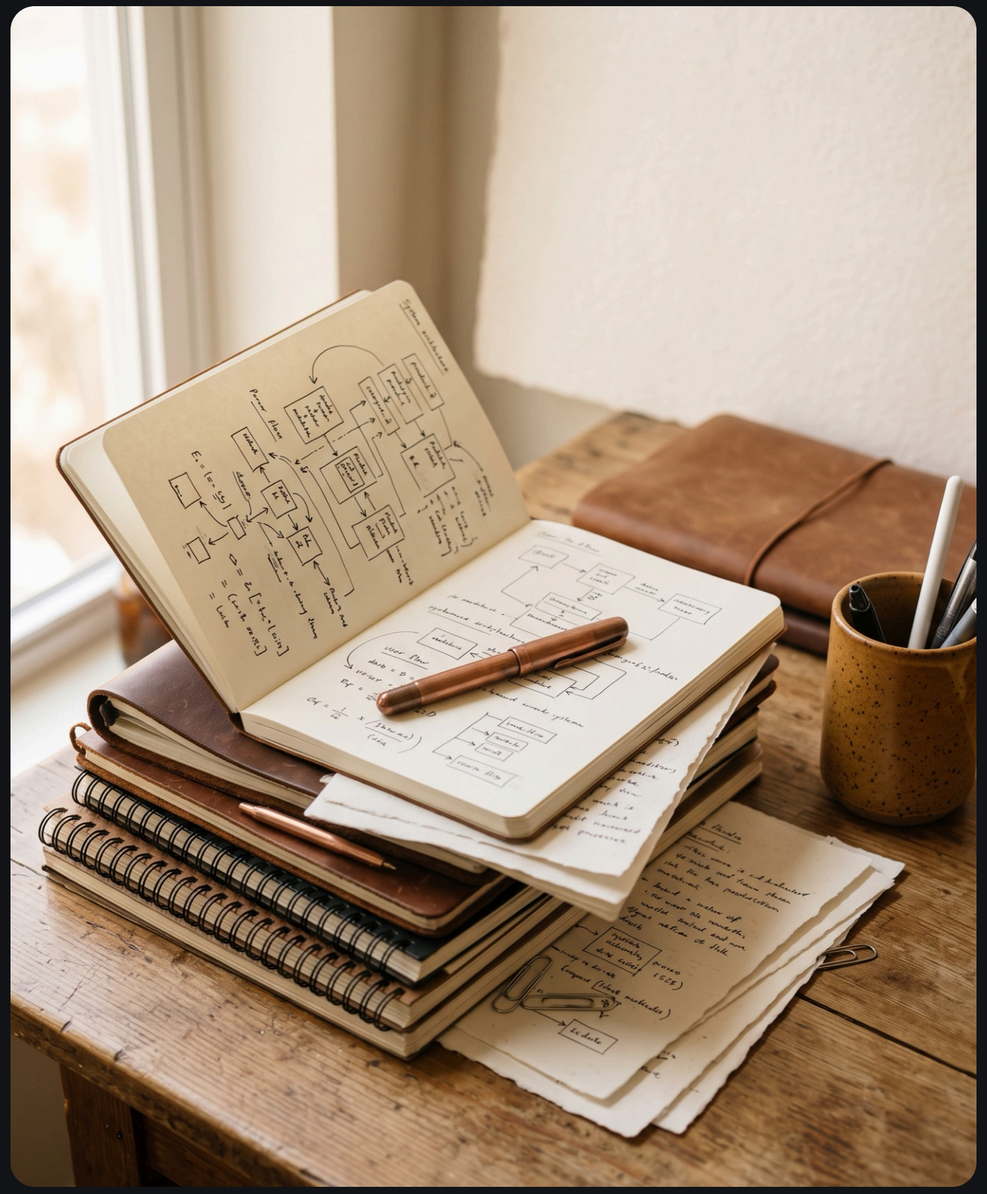USER GUIDE · LLM-WIKI

# A wiki your LLM *keeps for you.*

Index and search your personal documentation with Karpathy's LLM Wiki pattern: compiled once, kept current, cross-linked.

BikS2013 · field notes
Athens · October 2026



BikS2013

# Not retrieval at question time – *compilation* at ingest time.

01

## You curate the sources

Drop articles, notes, PDFs and DOCX files into `raw/`. They are never modified – this is your source of truth.

02

## The LLM writes the wiki

Every ingest produces a summary page and updates the entity and concept pages it touches, with wikilinks and citations.

03

## You ask, it compounds

Search locally, ask with citations, file good answers back as pages, and lint for contradictions and gaps.

Pattern: Andrej Karpathy, "LLM Wiki" (gist, April 2026). Implementation: github.com/biks2013-tools/llm-wiki.



00 · THE IDEA

# Three layers in one folder.

LAYER 1 · IMMUTABLE

## Raw sources

Your documents, exactly as you added them. Read, never written – except the markdown conversion placed beside a document or image.

```
raw/article.md
raw/report.pdf
raw/report.pdf.md
raw/assets/
```

LAYER 2 · LLM-OWNED

## The wiki

Interlinked pages – markdown, or HTML for the browser – the model creates and keeps consistent, plus a generated catalog and an append-only log.

```
wiki/index · wiki/log (.md|.html)
wiki/sources/ · entities/
wiki/concepts/ · syntheses/
wiki/lint/
```

LAYER 3 · THE CONTRACT

## The schema

A CLAUDE.md / AGENTS.md written by `init`: layout, page format, link rules and the ingest / query / lint workflows any agent must follow.

```
CLAUDE.md
AGENTS.md
.llm-wiki/ (tool state)
```

# 01

## Set it up once.

Install the CLI, choose the models, then create and register your wikis — markdown or HTML. Nothing is defaulted silently — every missing setting is a clear error.



# Install in three commands.

```
# requires Node 22+ and pnpm
git clone git@github.com:biks2013-tools/llm-wiki.git
cd llm-wiki
pnpm install
pnpm link:global      # build + put `llm-wiki` on your PATH

llm-wiki --version
llm-wiki guide        # the complete manual
```

- **An OS command:** the link points at the checkout, so `pnpm build` updates it; `pnpm unlink:global` removes it.
- **Built-in guide:** `llm-wiki guide <topic>` or `--json` – generated from the installed CLI, never stale.
- **Check the build** with `pnpm test`: 54 tests, no network needed. Dependencies are vetted and audited.

# Configure once — four tiers, *no* hidden defaults.

1 · LOWEST
**Shell env**
What your shell already exports

2
**~/.tool-agents/ llm-wiki/.env**
Your long-lived settings (0600)

3
**./.env**
Per-project overrides

4 · HIGHEST
**CLI flags**
--wiki, --model, …

| SETTING | PURPOSE |
|---|---|
| `LLM_WIKI_NAME / --wiki` | Which registered wiki · or `LLM_WIKI_ROOT / --root` |
| `LLM_WIKI_PROVIDER / _MODEL` | Chat model for ingest, query, lint --llm |
| `LLM_WIKI_EMBEDDING_PROVIDER / _MODEL` | Optional · enables vector and hybrid search |
| `LLM_WIKI_PARSER_PROVIDER / _MODEL` | Optional · converts PDF and DOCX originals |

```
# ~/.tool-agents/llm-wiki/.env
LLM_WIKI_PROVIDER=anthropic
LLM_WIKI_MODEL=claude-opus-5
ANTHROPIC_API_KEY=…

LLM_WIKI_EMBEDDING_PROVIDER=openai
LLM_WIKI_EMBEDDING_MODEL=text-embedding-3-small
OPENAI_API_KEY=…

LLM_WIKI_PARSER_PROVIDER=gemini
LLM_WIKI_PARSER_MODEL=gemini-3.6-pro
GOOGLE_API_KEY=…
```

`llm-wiki config` shows every resolved setting, its tier, and masks the keys.

# Eight providers, vendor-canonical variables.

| PROVIDER | VARIABLES IT READS | CHAT | EMBED | PDF NATIVELY |
|---|---|---|---|---|
| **openai** | `OPENAI_API_KEY` · `OPENAI_BASE_URL` · `OPENAI_ORG_ID` | yes | yes | yes |
| **anthropic** | `ANTHROPIC_API_KEY` · `ANTHROPIC_BASE_URL` | yes | — | yes |
| **gemini** | `GOOGLE_API_KEY (or GEMINI_API_KEY)` | yes | yes | yes |
| **azure-openai** | `AZURE_OPENAI_API_KEY` · `_ENDPOINT` · `_DEPLOYMENT` · `_API_VERSION` | yes | yes | yes |
| **azure-anthropic** | `AZURE_AI_INFERENCE_KEY` · `AZURE_AI_INFERENCE_ENDPOINT` | yes | — | yes |
| **ollama** | `OLLAMA_HOST` | yes | yes | text extracted locally |
| **litellm** | `LITELLM_PROXY_URL` · `LITELLM_MASTER_KEY` | yes | yes | yes |
| **mlx** | `OPENAI_BASE_URL (OpenAI-compatible server)` | yes | yes | text extracted locally |

Model names are yours to choose (e.g. claude-opus-5, gpt-5, gemini-3.6-flash, llama3). For azure-openai the chat model is the deployment in AZURE_OPENAI_DEPLOYMENT.

# Create the wiki: *init*.

```
$ llm-wiki init ~/notes-wiki --format markdown \
    --name notes --description "Personal notes"
initialised markdown wiki at /Users/you/notes-wiki
  created …/raw  raw/assets
  created …/wiki/sources  entities  concepts  syntheses  lint
  created …/.llm-wiki/wiki.json  CLAUDE.md  AGENTS.md
  created …/wiki/index.md  wiki/log.md
registered as "notes"
"notes" is now the current wiki

$ llm-wiki init ~/handbook --format html --name handbook
```

WHAT YOU GET

### A folder that is also a repo

Markdown for Obsidian and editors, or HTML pages that open in a browser – your choice per wiki. Put it under git; open it in Claude Code and the schema makes the agent its maintainer.

SAFE TO RE-RUN

### Existing files are kept

`init` refuses to overwrite a wiki that already has a schema; `--force` rewrites only CLAUDE.md and AGENTS.md.

# 02

## The daily loop.

Ingest a source, search the pages, ask a question, file the answer, lint now and then. Each step leaves the wiki richer than it found it.



# Ingest: one source in, *five to fifteen* pages touched.

```
$ llm-wiki ingest ~/Downloads/deploy-notes.md
reading related pages for raw/deploy-notes.md
asking anthropic/claude-opus-5 to compile raw/deploy-notes.md
# Deployment notes for the Ntakker API
Takeaways:
- Deploys to Azure Web App as a Docker container …
Pages written:
- replace wiki/sources/deploy-notes.md
- create wiki/entities/ntakker-api.md
- create wiki/entities/azure-blob-storage.md
- create wiki/concepts/configuration-management.md
  …

$ llm-wiki ingest                  # every pending source
$ llm-wiki ingest raw/x.md --dry-run
```

- **Context for the model:** the index plus the most related existing pages, found by search – so it updates pages instead of duplicating them.
- **After writing:** `index.md` is regenerated, the search index rebuilt, a log entry appended, and the source's hash recorded.
- **Idempotent:** an unchanged source is skipped; `--force` redoes it; `--dry-run` shows the plan only.
- **Never truncates:** a source above `--max-source-chars` is refused with a message, not silently cut.

# Search: local, instant, *three* modes.

BM25 · DEFAULT

## Keyword ranking

Okapi BM25 over heading-level chunks with a light stemmer. No model, no network, works the moment a page exists.

VECTOR

## Semantic similarity

Cosine over embeddings cached by chunk hash. Run `llm-wiki index --embed` once; only new or changed chunks are embedded afterwards.

HYBRID

## Both, fused

Reciprocal-rank fusion of the two lists. The default for `query` as soon as embeddings exist.

```
$ llm-wiki search "postgres docker"                    # bm25, wiki pages
$ llm-wiki search "how do I run the db" --mode hybrid --limit 5
$ llm-wiki search "psql" --layer all --json              # include raw/ sources, machine-readable
1. wiki/entities/postgres.md › Postgres  (0.727)
   Postgres runs in a Docker container on port 5432. See [[docker]] …
```

# Query: answers with citations — *filed back* on request.

```
$ llm-wiki query "Where does the API read its config?" --file
### Where the API reads configuration
The [[entities/ntakker-api|Ntakker API]] fetches its configuration
at startup from [[entities/azure-blob-storage]] in the `app-config`
container ([[sources/deploy-notes]]).

### What remains undecided
Whether to move to Azure App Configuration ([[entities/giorgos]]).

### Sources
- [[sources/deploy-notes]] · [[entities/ntakker-api]] · …
filed as wiki/syntheses/where-does-the-api-read-its-config….md
```

- **Every claim cites a page.** If the wiki lacks the answer, the model says what is missing instead of inventing it.
- `--file` stores the answer under `wiki/syntheses/` with frontmatter, updates the index and logs it.
- **Several wikis at once:** `--wikis a,b` or `--all-wikis`; citations read `[[wiki:page]]`.
- **Tune the context:** `--limit`, `--pages`, `--context-chars`, `--mode`.

# Lint: a health check in *two* passes.

STRUCTURAL · NO LLM

## Deterministic findings

Dangling wikilinks · orphan pages · pages without outbound links · missing frontmatter or summary · basename collisions · stale `index.md` · sources in `raw/` not yet ingested · PDF/DOCX originals not yet converted.

SEMANTIC · --LLM

## What only a reader can see

Contradictions between pages · claims superseded by newer sources · concepts mentioned repeatedly but lacking a page · missing cross-references · data gaps · suggested questions and sources.

```
$ llm-wiki lint                    # structural, exit 1 on errors
$ llm-wiki lint --fix              # also regenerate index.md and the search index
$ llm-wiki lint --llm --report     # semantic pass, filed as wiki/lint/<date>.md and logged
```

# Two special files: *index.md* and *log.md*.

## index.md — what is in the wiki

A catalog of every page grouped by type, each with a link, its one-line `summary`, its date and source count. Generated from page frontmatter by `llm-wiki index` — never hand-edit it; edit the frontmatter instead.

```
## Entities (6)
- [[azure-blob-storage|Azure Blob Storage]] — Object storage
  used for runtime configuration. _(2026-09-12, 1 source)_
- [[ntakker-api|Ntakker API]] — Backend API deployed to
  Azure Web App in Docker. _(2026-09-12, 1 source)_
```

## log.md — what happened, when

Append-only. Every entry starts with the same prefix, so the log is grep-able: `grep "^## \[" log.md | tail -5`. The CLI reads it too: `llm-wiki log --last 5 --kind ingest`.

```
## [2026-09-12] ingest | Deployment notes for the Ntakker API
Source: raw/deploy-notes.md
Pages touched: … · Takeaways: …

## [2026-09-12] query | Where does the API read its config…
Filed as [[syntheses/where-does-the-api-…]].
```

In HTML wikis they are `index.html` and `log.html`. `llm-wiki index --raw` also indexes raw sources for search; `--embed` computes missing embeddings.

# 03

## Documents, originals intact.

PDF, Word, PowerPoint, Excel and images become markdown – by a dedicated parser model, or locally for spreadsheets. The original stays untouched in `raw/` and every derived page points back to it.



# A separate *parser* model, chosen for the job.

1 · YOU ADD

## raw/report.pdf

A PDF, DOCX, PPTX or image, copied into `raw/` if it was elsewhere. It is never modified. (XLSX skips the model — see slide 18.)

→

2 · PARSER LLM

## faithful markdown

Headings, lists, tables, figures as bracketed descriptions. Nothing summarised, nothing dropped. Set by `LLM_WIKI_PARSER_PROVIDER / _MODEL`.

→

3 · THE WIKI READS

## raw/report.pdf.md

Written beside the original with provenance frontmatter, then ingested like any other source.

```
$ llm-wiki ingest raw/report.pdf            # convert, then ingest
$ llm-wiki parse raw/q3-review.pptx         # convert only — review raw/q3-review.pptx.md first
$ llm-wiki ingest raw/report.pdf --reparse  # redo the conversion; cached by hash otherwise
```

# Provenance travels with every page.

```
# raw/report.pdf.md
---
title: Quarterly Report
original: raw/report.pdf
original_type: PDF
original_sha256: 3f2a9c…
parsed_by: gemini/gemini-3.6-pro
parsed_at: 2026-09-12
---
> **Converted from the original PDF document
> [[raw/report.pdf]]** (412 KB) by gemini/gemini-3.6-pro
> on 2026-09-12. The original is kept unmodified in raw/.

# Quarterly Report
…
```

ON THE WIKI PAGE TOO

### sources/quarterly-report.md

Gets `original: raw/report.pdf` in its frontmatter and a link to the original in its body – stamped by the tool, whatever the model wrote.

NATIVE VS EXTRACTED

### Scanned PDFs work

Anthropic, Gemini, OpenAI, Azure and LiteLLM receive the PDF itself. Ollama and MLX receive text extracted with pdf.js – a scanned page yields nothing there, and the tool says so.

# Slides, sheets and pictures, *too*.

PPTX · PARSER MODEL

## One section per slide

Titles, text in reading order, tables, the numbers behind charts and speaker notes are read locally; the slide pictures go along, so the model transcribes screenshots and diagrams.

XLSX · NO LLM

## Exact tables

One markdown table per sheet, converted locally: stored values, ISO dates, percentages, formulas as their result. Not a single number passes through a model.

PNG · JPG · GIF · WEBP

## Read by a vision model

Whiteboards, screenshots, scanned notes: every visible word transcribed, charts as data tables, diagrams and photos described.

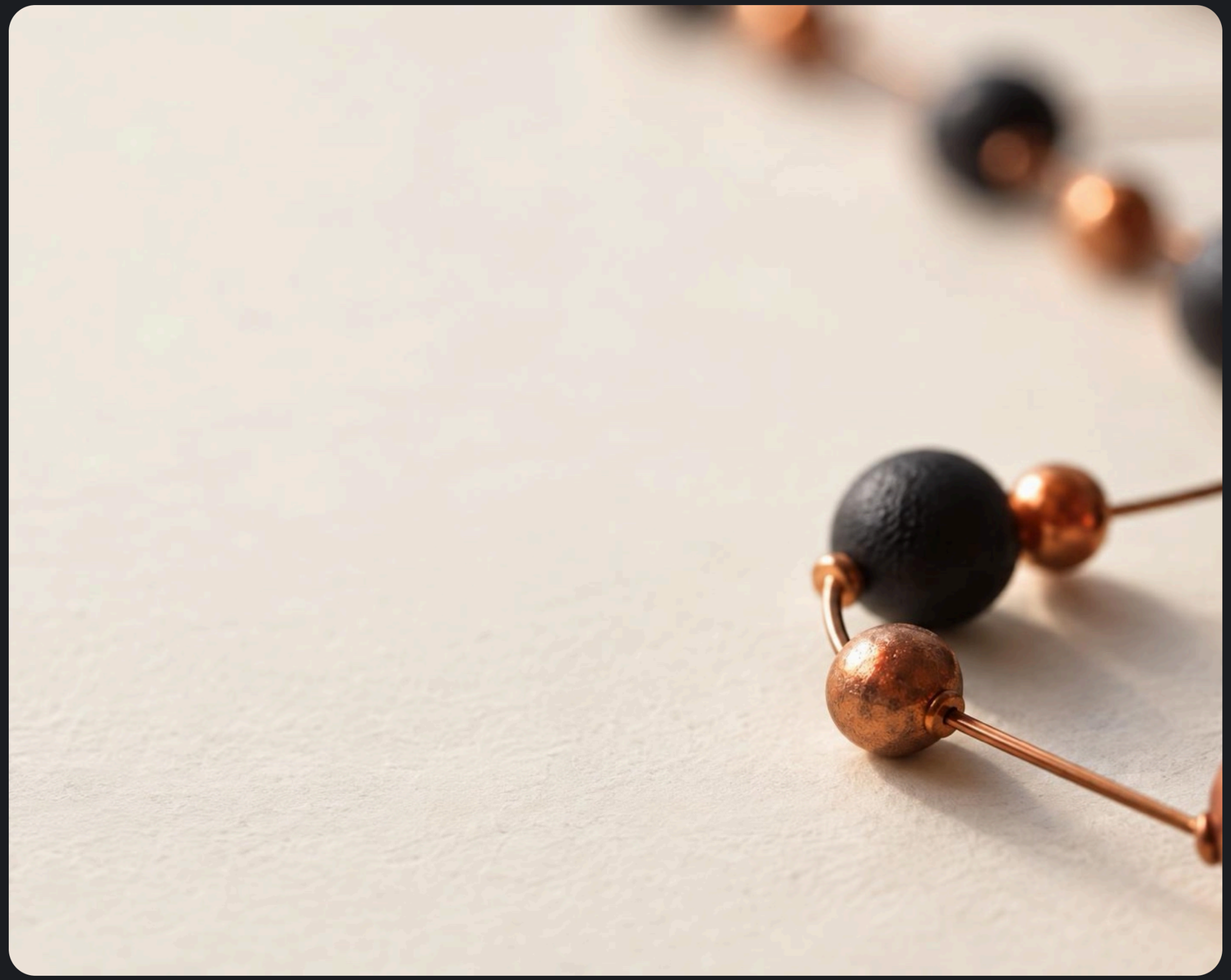
```
$ llm-wiki --wiki club ingest ~/Downloads/q3-review.pptx   # 5 slides, 1 picture sent
$ llm-wiki --wiki club parse budget.xlsx whiteboard.jpg     # review raw/*.md before ingesting
```

Refused with a hint: .doc .ppt .xls (save as the x format), .heic .tif (convert to JPEG/PNG). Limits per format: `llm-wiki guide layout`.

# 04

## Let the agent do the bookkeeping.

Obsidian is the IDE, the LLM is the programmer, the wiki is the codebase. The schema and the MCP server make that literal.



# The schema turns any coding agent into the *maintainer*.

§1

### Layout & page format

Where each page type lives, the frontmatter keys, kebab-case slugs, wikilink conventions, citing `raw/` for every non-obvious claim.

§2

### Operations

Step-by-step ingest, query and lint workflows — including "run `llm-wiki index` after editing pages" and how to flag a contradiction.

§3

### Working style

Never touch `raw/`; keep pages atomic; prefer updating over duplicating; mark possible duplicates for the next lint pass.

```
$ cd ~/notes-wiki && claude          # CLAUDE.md is read automatically; AGENTS.md serves Codex and others
> ingest raw/podcast-notes.md and tell me what changed in the overview
```

# The MCP server: native tools for the agent.

```
$ claude mcp add llm-wiki -- llm-wiki mcp

# or run it yourself for any MCP client
$ llm-wiki mcp                  # stdio
```

One server serves **every registered wiki**: each tool takes an optional `wiki` argument (default: the selected wiki), and the registry is re-read on every call.

| TOOL | DOES |
|---|---|
| `wiki_list_wikis` | the registered wikis and their status |
| `wiki_search` | one, several or all wikis |
| `wiki_read_page` | a page, or index / log |
| `wiki_list_pages` | id, type, title, summary; filter by type |
| `wiki_write_page` | create / replace / append; reindexes |
| `wiki_append_log` | an entry in the standard format |
| `wiki_lint` | the structural health check |
| `wiki_reindex` | index.md + search index |

# What a page looks like.

```
---
title: Ada Lovelace
type: entity          # entity | concept | source | synthesis
summary: One sentence shown verbatim in index.md.
tags: [mathematics, computing]
created: 2026-04-02
updated: 2026-04-09
sources: [raw/some-article.md]
aliases: [Countess of Lovelace]
---
# Ada Lovelace

Worked with [[charles-babbage]] on the
[[concepts/analytical-engine|Analytical Engine]]
([[sources/some-article]]).

> **Contradiction:** [[sources/a]] dates the notes to 1842,
> [[sources/b]] to 1843.
```

- **Frontmatter feeds the index:** `title`, `summary`, `type`, `updated`, `sources`.
- **Wikilinks** by basename when unique, by `folder/slug` when not; lint flags collisions and dangling links.
- **Contradictions are callouts**, never silent overwrites.
- **After hand edits**, run `llm-wiki index`.
- **HTML wikis** store the same metadata as JSON in the page head and the body in `<main>`; the tool writes them from this same form.

# A worked example, from one small note.

8

## pages from a four-line deployment note

One source page, six entities (the API, the team member, three Azure services, Docker) and one concept – all cross-linked, all indexed.

| STEP | RESULT |
|---|---|
| **ingest** | 8 pages written · index and log updated · 17 chunks indexed |
| **index --embed** | 17 embeddings computed once, cached by hash |
| **search --mode hybrid** | "where does production config live" → the configuration-management page, first hit |
| **query --file** | Answer cited 5 pages · filed under `syntheses/` |
| **lint --llm --report** | Suggested a missing page (Azure App Configuration), one missing cross-reference, three next questions |

Run on 2026-09-12 with gemini-3.6-flash as chat and parser model and text-embedding-3-small for embeddings.

# When it stops, it tells you why.

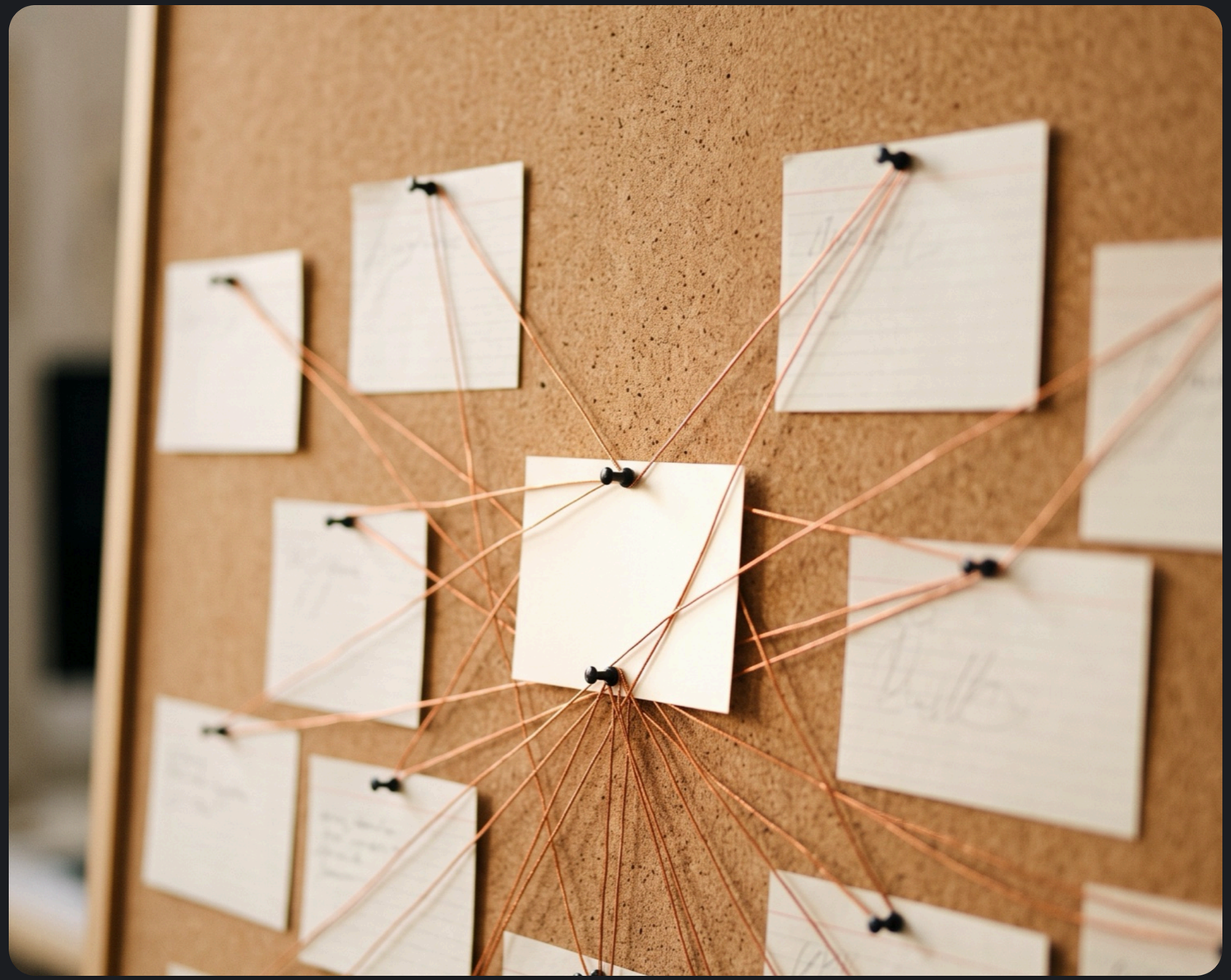
| MESSAGE | WHAT TO DO |
|---|---|
| `No wiki selected. / No wiki named "x" is registered.` | Pass `--wiki <name>` (the message lists them), or `llm-wiki wikis use <name>`. |
| `Missing required configuration setting LLM_WIKI_…` | Add it to `~/.tool-agents/llm-wiki/.env` or the shell – there are no defaults. |
| `… is not an initialised wiki (missing raw/)` | Run `llm-wiki init <dir>` first. |
| `… original and needs the parser LLM · … cannot read images` | Set `LLM_WIKI_PARSER_PROVIDER` and `_MODEL` – a model with vision for images. |
| `Search mode "hybrid" needs an embedding provider / No embeddings are stored yet` | Set the embedding settings, then `llm-wiki index --embed`. |
| `… hit the N-token output limit before finishing` | Raise `--max-tokens` (ingest, query, lint) or `--parse-max-tokens`. |
| `… is above the --max-source-chars / --max-bytes limit` | Split the source or raise the limit – the tool never truncates silently. |

Add `--verbose` to any command for a stack trace. Exit codes: 2 configuration, 3 LLM request, 1 anything else.

# 05

## Many wikis, two formats.

Keep one wiki per subject, pick the one each command works on, ask across them – and read them as markdown or in the browser.



# A registry of wikis — every command names *one*.

```
$ llm-wiki wikis list
* notes      ~/notes-wiki
    Personal notes
    markdown, 42 page(s), 0 pending source(s)
  handbook   ~/handbook
    Team handbook, read in a browser
    html, 118 page(s), 2 pending source(s)

$ llm-wiki --wiki handbook ingest ~/Downloads/onboarding.pdf
$ llm-wiki wikis use handbook       # change the default
$ llm-wiki wikis add old ~/old-wiki  # register an existing one
```

HOW A COMMAND PICKS ITS WIKI

1. **--wiki <name> or --root <dir>**
   or `LLM_WIKI_NAME` / `LLM_WIKI_ROOT`
2. **the current wiki**
   set with `wikis use`
3. **ask on the terminal**
   a numbered prompt; `--pick` forces it

Otherwise the command stops and lists the wikis. Registry: `~/.tool-agents/llm-wiki/wikis.json`.

# Search and ask *across* wikis.

```
$ llm-wiki search "enigma" --all-wikis
1. [notes] wiki/entities/bombe.md › Bombe  (0.033)
2. [site]  wiki/concepts/enigma-cipher.html › Enigma Cipher  (0.032)

$ llm-wiki query "How did the Polish work relate to Turing's?" \
    --wikis site,notes
The Polish bomba (1938) was the basis of Turing's British
Bombe [[notes:entities/bombe]], designed at Bletchley Park
[[site:entities/bletchley-park]] …

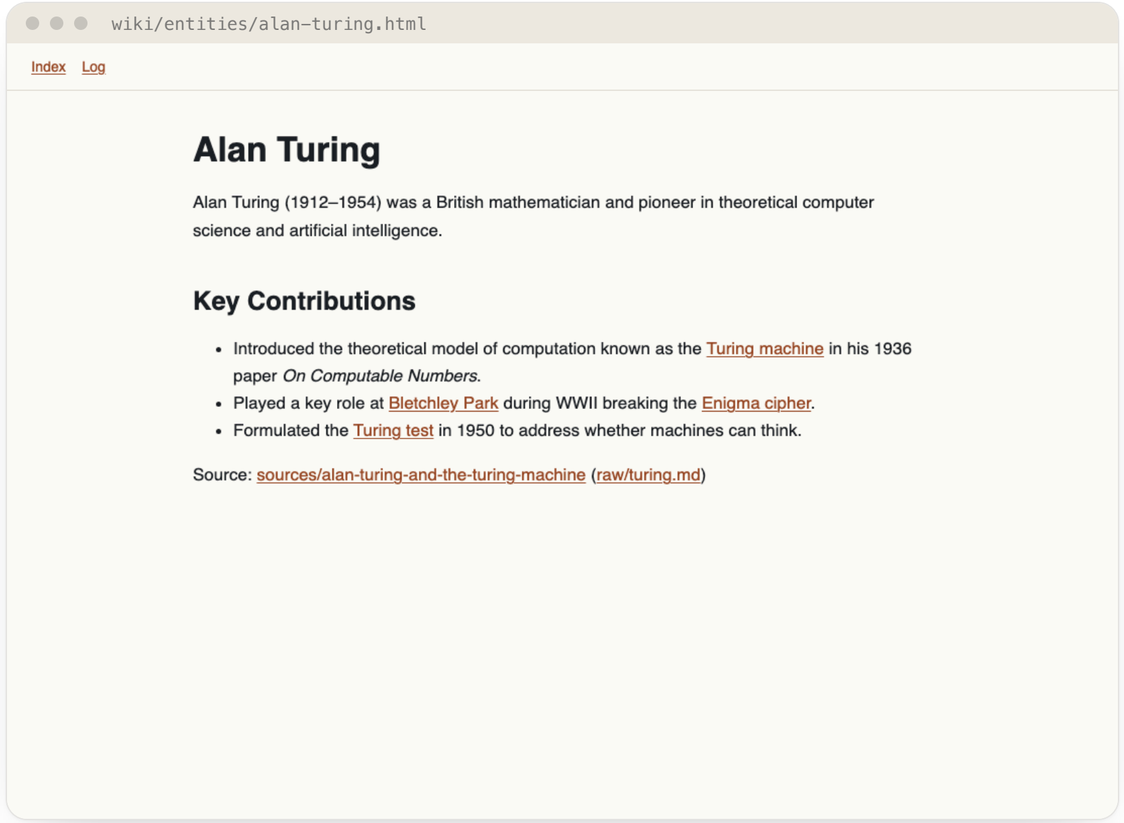
### Sources
notes · [[notes:entities/bombe]] · [[notes:entities/marian-rejewski]]
site  · [[site:entities/bletchley-park]] · …
```

- **Fair ranking:** each wiki is searched on its own index; the lists are merged by reciprocal-rank fusion.
- **One budget:** the best pages from all wikis share `--pages` and `--context-chars`.
- **Citations name the wiki** and every wiki consulted gets a log entry. A cross-wiki answer is not filed – re-ask one wiki with `--file`.

# Markdown or HTML — *per wiki*.

| | MARKDOWN | HTML |
|---|---|---|
| **Page** | `wiki/<id>.md` | `wiki/<id>.html` |
| **Metadata** | YAML frontmatter | JSON in the page head |
| **Links** | `[[ada-lovelace]]` | relative `<a href>` |
| **Read in** | Obsidian, editors, agents | any browser, offline |

- **Chosen at `init --format`** – required, never defaulted.
- **Same commands:** the LLM writes frontmatter + body; the tool builds the HTML, turns `[[links]]` into real links and disables scripts.



# Convert a wiki; let Claude *drive* it.

CONVERT

## Markdown → HTML, in place

Ids, metadata and links survive; log, index and search are rebuilt. No LLM. The markdown is kept in a backup folder; a customised `CLAUDE.md` keeps your own sections.

GUIDE

## The manual, built in

`llm-wiki guide` prints eleven topics, with a command reference generated from the CLI itself; `--json` for tools.

SKILL

## A Claude skill

`pnpm skill:install` installs it for Claude Code: Claude picks the wiki, uses `--json` and keeps to the guardrails.

```
$ llm-wiki --wiki notes convert --to html --dry-run   # pages, unresolved links, schema plan
$ llm-wiki --wiki notes convert --to html             # then: open ~/notes-wiki/wiki/index.html
```

CHEAT SHEET

# Your documentation, *compiled.*

```
llm-wiki init ~/notes --format markdown|html --name notes
llm-wiki --wiki notes ingest <file>     # text, PDF, DOCX, PPTX, XLSX or an image
llm-wiki --wiki notes search "<terms>"   # --all-wikis · --mode hybrid after `index --embed`
llm-wiki --wiki notes query "<q>" --file # cited answer, filed · --wikis a,b to ask several
llm-wiki --wiki notes lint --llm         # contradictions, gaps, next questions
llm-wiki --wiki notes convert --to html  # make a markdown wiki browsable
llm-wiki guide  ·  llm-wiki mcp          # the manual · tools for your agent
```

BikS2013

github.com/biks2013-tools/llm-wiki
Pattern: Karpathy, LLM Wiki, 2026